

Facial Tracking (No-PTZ)

1. Experiment Objective

OpenCV Haar Cascade is used to detect faces, and the car rotates to track so that the face stays at the center of the frame. It supports Tracking mode (rotation only) and Follow mode (rotation + distance keeping).

2. Experiment Principle

1. Keyword Explanation

Keyword	Description
Face Detection	Uses the OpenCV Haar Cascade classifier to detect frontal face regions
Haar Cascade	A cascade classifier based on Haar-like features that rapidly detects faces through a pre-trained XML model
Centroid Deviation	The horizontal pixel offset between the face box center and the frame center, used as the PD controller input
PD Control	Proportional-derivative controller that computes the body angular velocity based on the centroid deviation and its rate of change
Deadband	No steering is triggered while the target is within a certain range around the frame center, avoiding frequent steering from small jitter
Distance Following	Controls the car to move forward/backward based on the size change of the face box width in the frame to keep a constant distance
QoS	best_effort (image input + cmd_vel downlink), depth=1

2. Technical Architecture

/camera/image_raw (BEST_EFFORT)	
→ face_tracker_node	
└─ Image processing timer (15Hz): image conversion → grayscale → Haar Cascade detection → largest face selection	
└─ Body control timer (10Hz): pixel error → PD angular velocity + face width distance following → /cmd_vel (BEST_EFFORT)	

3. Core Topic Quick Reference

Firmware Uplink (Sensor Data)

Topic	Message Type	QoS	Description
/camera/image_raw	sensor_msgs/Image	best_effort	ESP32 camera raw image frame

Firmware Downlink (Control Commands)

Topic	Message Type	QoS	Field	Range	Direction
/cmd_vel	geometry_msgs/Twist	best_effort + keep_last(1)	linear.x	-0.16~0.16 m/s	$\pm$ forward, $\pm$ backward
			angular.z	-1.00~1.00 rad/s	$\pm$ turn left, $\pm$ turn right

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	✗
No-PTZ version	☑
Required peripheral	ESP32 camera (bracket-mounted)

2. Start the Agent + Camera + Joint Model (Terminal 1)

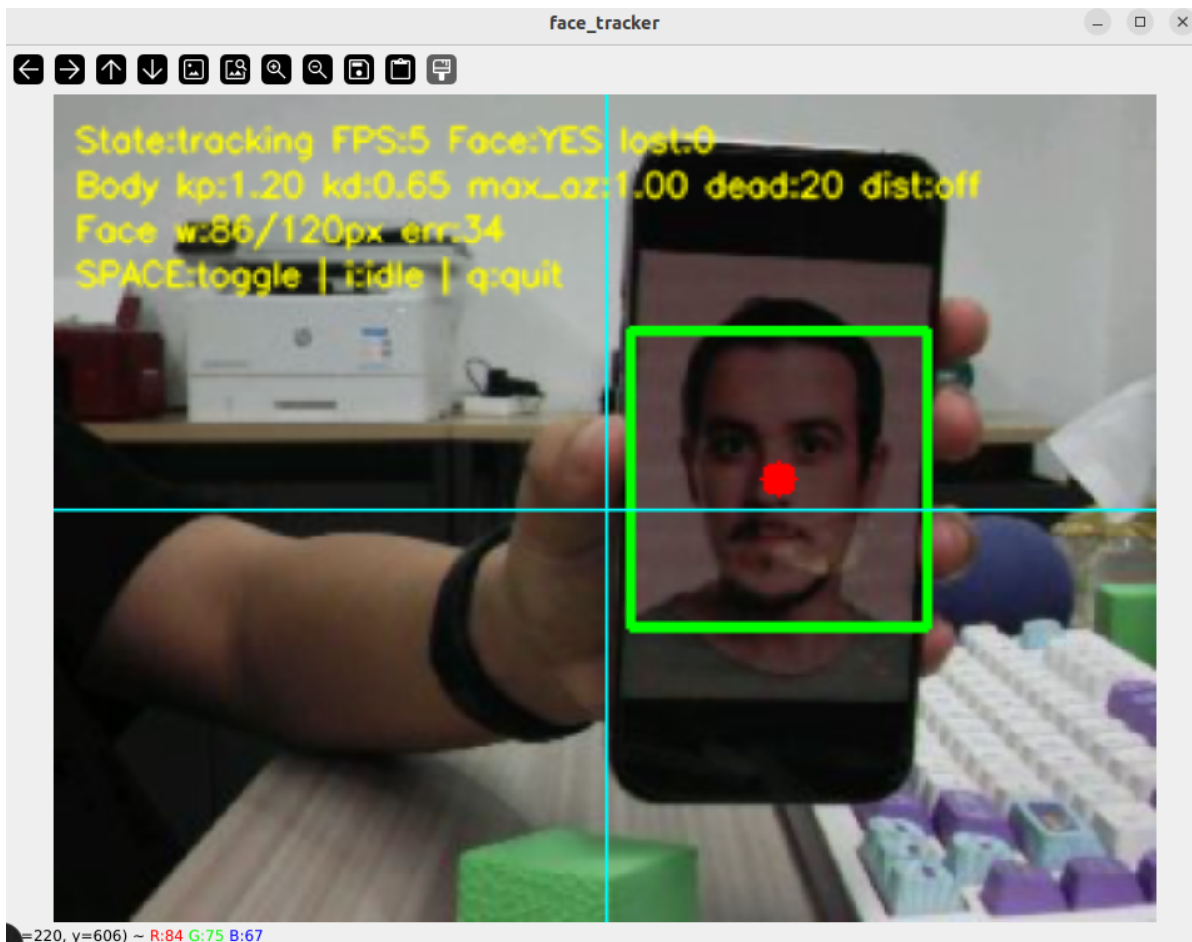
```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom: $ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info | UDPv4AgentLinux.cpp | Init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment can1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment can2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment can3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start Facial Tracking (Terminal 2)

Tracking mode (rotation tracking only):

```
ros2 launch microros_v2_no_ptz_visual_control_pkg face_tracker.launch.py
```



4. Operation Instructions

Key	Function
Space	Toggle tracking/standby state
i / I	Switch to the standby (IDLE) state
q / Q / ESC	Exit the program

4. Experiment Phenomenon

In Tracking mode:

- When the face moves left or right in the frame, the car automatically rotates to track it, keeping the face at the center of the frame
- When multiple faces are present, the largest one is automatically selected as the tracking target

In Follow mode:

- While rotating to track, the car adjusts its distance to the target based on the width of the face box

When the target is lost:

- After the face moves out of the frame or turns away, the car stops moving
- When the face reappears, tracking resumes automatically

5. Code Explanation

Source path:

- Launch (track):
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/face_tracker.launch.py`
- Launch (follow):
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/face_follow.launch.py`
- Node source:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/face_tracker_node.py`

1. Core Node Analysis

`face_tracker_node` uses the OpenCV Haar Cascade classifier to detect faces. Core workflow:

- **Image processing** (15Hz): raw Image frame → cv_bridge conversion → grayscale → `face_cascade.detectMultiScale()` → select the largest face as the target
- **Body control** (10Hz): deviation between the face center x-coordinate and the image center → PD computes angular velocity; in Follow mode the face width deviation → PD computes linear velocity

Constructor — load the Haar Cascade model, create publishers/subscribers:

```
# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/face_tracker_node.py

def __init__(self) -> None:
    super().__init__(DEFAULT_NODE_NAME)
    self._tgt_lock = threading.Lock()
    self._latest_frame: Optional[np.ndarray] = None

    self._state = STATE_IDLE
    self._has_target = False
    self._lost_frames = 0
    self._target_x: Optional[float] = None
    self._target_y: Optional[float] = None
    self._target_w: Optional[int] = None # Face box width (used for distance following)

    self._declare_params()
    self._load_params()

    # Load the OpenCV Haar Cascade face detection model
    cascade_path = cv.data.harcascades + "haarcascade_frontalface_default.xml"
    self._face_cascade = cv.CascadeClassifier(cascade_path)

    best_effort = QoSProfile(
        history=QoSHistoryPolicy.KEEP_LAST, depth=1,
        reliability=QoSReliabilityPolicy.BEST_EFFORT,
        durability=QoSDurabilityPolicy.VOLATILE,
    )
    self._img_sub = self.create_subscription(
        Image, self._img_topic, self._on_image, best_effort)
    self._cmd_pub = self.create_publisher(Twist, self._cmd_topic, best_effort)
```

```

        self._img_timer = self.create_timer(
            1.0 / max(float(self._img_rate), MINIMUM_TIMER_RATE_HZ),
self._process_image)
        self._body_timer = self.create_timer(
            1.0 / max(float(self._body_rate), MINIMUM_TIMER_RATE_HZ),
self._body_tick)

```

Body control callback — facial PD tracking:

```

# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visua
l_control_pkg/face_tracker_node.py

def _body_tick(self) -> None:
    """Body control timer (10Hz)."""
    if self._state != STATE_TRACKING or self._shutdown:
        self._pub_zero()
        return

    with self._tgt_lock:
        has = self._has_target
        tx = self._target_x
        tw = self._target_w

    if not has or tx is None:
        self._pub_zero()
        return

    # Horizontal pixel error → PD angular velocity
    cx = self._img_w * 0.5
    err_x = tx - cx
    if abs(err_x) <= self._deadband_x:
        az = 0.0
    else:
        now = time.perf_counter()
        dt = now - self._prev_t if self._prev_t > 0 else 1.0 / self._body_rate
        dt = max(dt, 1e-3)
        d_err = (err_x - self._prev_err) / dt
        self._prev_err = err_x; self._prev_t = now
        az = self._angular_kp * err_x + self._angular_kd * d_err
        az = clamp(az, -self._max_az, self._max_az)
        if self._reverse_body:
            az = -az

    cmd = Twist()
    cmd.angular.z = az

    # Distance following: face width deviation → PD linear velocity
    if self._dist_follow and tw is not None:
        err_fw = self._target_face_w - tw
        if abs(err_fw) > self._dist_tol_fw:
            now = time.perf_counter()
            dt = now - self._prev_dt if self._prev_dt > 0 else 1.0 /
self._body_rate
            dt = max(dt, 1e-3)
            d_err_fw = (err_fw - self._prev_derr) / dt

```

```

self._prev_derr = err_fw; self._prev_dt = now
lx = self._dist_kp * err_fw + self._dist_kd * d_err_fw
lx = clamp(lx, -self._max_lx, self._max_lx)
if self._reverse_linear:
    lx = -lx
cmd.linear.x = lx

self._cmd_pub.publish(cmd)

```

Key logic: `haarcascade_frontalface_default.xml` only detects frontal faces (low detection rate for side faces). `detectMultiScale` returns all face rectangles, and the one with the largest area is taken as the tracking target.

2. Key Parameters

Parameter definition file:

- Launch:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/face_tracker.launch.py`
- Node:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/face_tracker_node.py`

Parameter	Type	Default	Description
<code>face_min_width</code>	int	80 px	Minimum face width; detections below this value are ignored
<code>face_min_height</code>	int	80 px	Minimum face height
<code>deadband_x_pixel</code>	float	20.0 px	Horizontal deadband
<code>angular_kp</code>	float	1.2	Body angle PD proportional coefficient
<code>angular_kd</code>	float	0.65	Body angle PD derivative coefficient
<code>max_angular_z</code>	float	1.0 rad/s	Maximum body steering speed
<code>lost_frame_threshold</code>	int	5	Consecutive lost-frame tolerance threshold
<code>enable_distance_follow</code>	bool	false	Whether to enable distance following
<code>target_face_width</code>	float	120.0 px	Target face width for distance following
<code>distance_tolerance_fw</code>	float	12.0 px	Distance-following deadband
<code>distance_kp</code>	float	0.012	Distance PD proportional coefficient
<code>distance_kd</code>	float	0.003	Distance PD derivative coefficient
<code>max_linear_x</code>	float	0.16 m/s	Maximum forward/backward speed

3. Node Description

Node	Package	Description
<code>face_tracker_node</code>	<code>microros_v2_no_ptz_visual_control_pkg</code>	Haar Cascade face detection + tracking with dual timers + PD body control

6. Common Problems

Problem	Cause	Solution
Side faces are not tracked	Haar Cascade mainly detects frontal faces	Keep the face facing the camera
Face is not detected	Insufficient lighting or the distance is too far	Improve ambient lighting and adjust <code>face_min_width</code>
Tracking jitter	Deadband too small or PD coefficients too large	Increase <code>deadband_x_pixel</code> or lower <code>angular_kp</code>
The car keeps turning back and forth	Multiple faces cause the detection target to switch frequently	Ensure only one face is in view